

# ASSIGNMENT\_01

2303A510H5

Batch\_30

Parimala

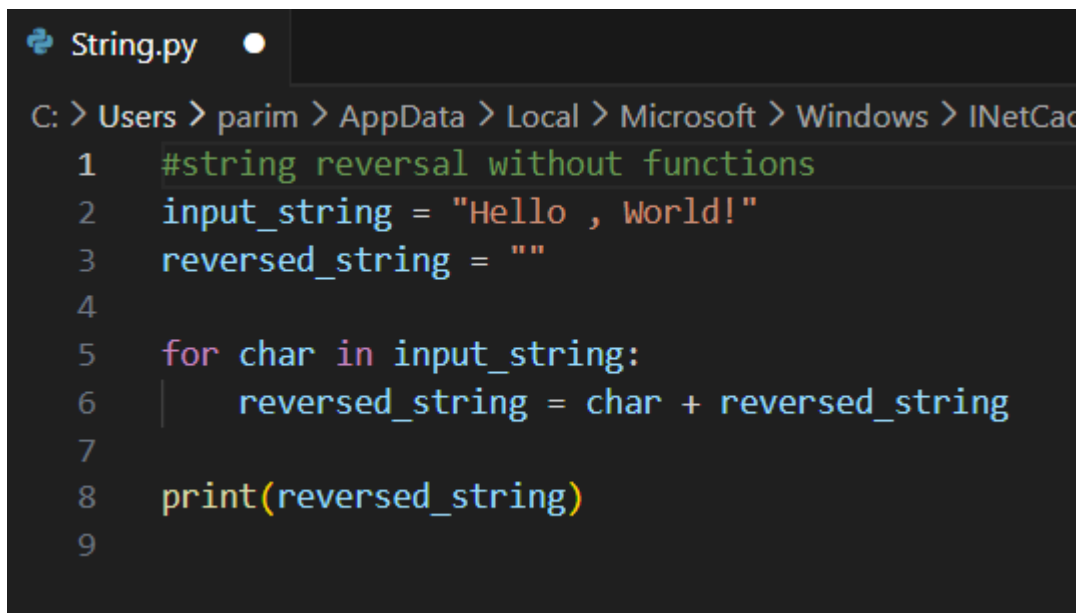
## Task\_01

AI-Generated Logic Without Modularization (String Reversal Without Functions)

Prompt:

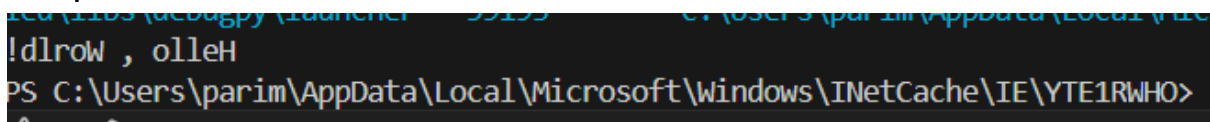
Generate a code for AI-Generated Logic Without Modularization (String Reversal Without Functions)

Code:



```
String.py
C: > Users > parim > AppData > Local > Microsoft > Windows > INetCac
1  #string reversal without functions
2  input_string = "Hello , World!"
3  reversed_string = ""
4
5  for char in input_string:
6      reversed_string = char + reversed_string
7
8  print(reversed_string)
9
```

Output:



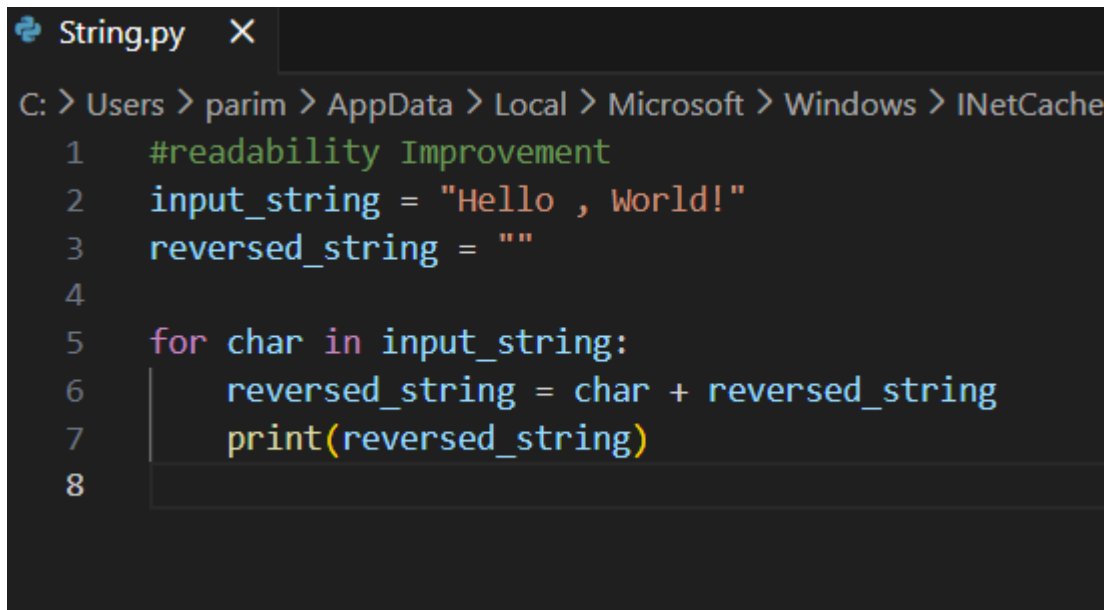
```
!dlrow , olleH
PS C:\Users\parim\AppData\Local\Microsoft\Windows\INetCache\IE\YTE1RWHO>
```

## TASK – 2: Efficiency & Logic Optimization (Readability Improvement)

Prompt:

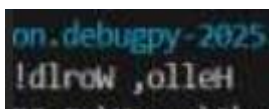
Generate a code for Efficiency & Logic Optimization (Readability Improvement)

Code:



```
String.py X
C: > Users > parim > AppData > Local > Microsoft > Windows > INetCache
1  #readability Improvement
2  input_string = "Hello , World!"
3  reversed_string = ""
4
5  for char in input_string:
6      reversed_string = char + reversed_string
7      print(reversed_string)
8
```

Output:



## Task – 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Prompt :

Generate a code for Modular Design Using AI Assistance (String Reversal Using Functions)

Code:

```
# String Reversal Using Functions
def reverse_string(s):
    reversed_s = ""
    for char in s:
        reversed_s = char + reversed_s
    return reversed_s
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(reversed_string)
```

Output:

```
PS C:\Users\parim> & 'c:\Users\parim\AppData\Local\Programs\Python\Python312\python32-x64\bundled\libs\debugpy\launcher' '58650' '--' 'c:\Users\parim\# String Reversal'
!dlrow ,olleH
PS C:\Users\parim>
```

Task – 4: Comparative Analysis – Procedural vs Modular Approach  
(With vs Without Functions)

Prompt :

Generate a code for Comparative Analysis – Procedural vs Modular  
Approach (With vs Without  
Functions)

Code:

```
# With vs Without Functions
def reverse_string(s):
    reversed_s = ""
    for char in s:
        reversed_s = char + reversed_s
    return reversed_s
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(reversed_string)
```

Output:

```
PS C:\Users\parim> c:; cd 'c:\Users\parim'; & 'c:\Users\parim\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\parim\AppData\Local\Programs\Python\Python310\Scripts\debugpy-launcher' '50753' '--' 'c:\Users\parim\AppData\Local\Programs\Python\Python310\python.exe' 'c:\Users\parim\AppData\Local\Programs\Python\Python310\Scripts\debugpy-launcher'
!dlrow ,olleH
PS C:\Users\parim>
```

Task – 5: AI-Generated Iterative vs Recursive Fibonacci Approaches  
(Different Algorithmic Approaches  
to String Reversal)

Prompt :

Generate a code for AI-Generated Iterative vs Recursive Fibonacci  
Approaches (Different Algorithmic  
Approaches to String Reversal)

Code:

```

# Different Algorithmic Approaches to String Reversal
# Using Slicing
input_string = "Hello, World!"
reversed_string = input_string[::-1]
print(reversed_string)

# Using reversed() Built-in
input_string = "Hello, World!"
reversed_string = "".join(reversed(input_string))
print(reversed_string)

# Using Recursion
def reverse_string_recursive(s):
    if len(s) == 0:
        return s
    return reverse_string_recursive(s[1:]) + s[0]

input_string = "Hello, World!"
reversed_string = reverse_string_recursive(input_string)
print(reversed_string)

# Using Stack
def reverse_string_stack(s):
    stack = [char for char in s]
    reversed_s = ""
    while stack:
        reversed_s += stack.pop()
    return reversed_s

input_string = "Hello, World!"
reversed_string = reverse_string_stack(input_string)
print(reversed_string)

```

Output:

```
PS C:\Users\parim> c:; cd "c:\Users\parim"; & "c:\Users\parim\AppData\Local\Programs\Python\Python311\python.exe" "c:\Users\parim\AppData\Local\Programs\Python\Python311\Scripts\python.exe" "c:\Users\parim\AppData\Local\Programs\Python\Python311\Scripts\debugpy-launcher.exe" -- "c:\Users\parim\AppData\Local\Programs\Python\Python311\Scripts\python.exe" "c:\Users\parim\AppData\Local\Programs\Python\Python311\Scripts\python.exe"
!dlrow ,olleH
!dlrow ,olleH
!dlrow ,olleH
!dlrow ,olleH
PS C:\Users\parim>
```